

EVA Engineering Spec — v0.3 Research Node

Stage 1 output — Architect Agent

Status: DRAFT — pending `core.py` confirmation (see Open Questions)

Legend

-  **CONFIRMED** — verified from actual running code (`api.py`)
 -  **PLANNED** — from the v0.3 Engineering Handoff Package, not yet verified in code
 -  **CONFLICT** — handoff doc says one thing, real code does another (real code wins per the One Rule)
 -  **OPEN** — unknown until `core.py` (and `tools/gateway.py` , `models.py`) are provided
-

1. Product Target

 EVA Research Node: a private research assistant with controlled, gated agents — not full autonomy.

Confirmed end-to-end flow (matches both `api.py` routes and the handoff doc):

1. User submits a mission (objective, optional source text or sources)
2. Pixie converts the request into a structured goal ( *not yet seen in code — only referenced in handoff doc*)
3. EVA Core creates task(s)
4. Sentinel authorizes tool/task usage (`core.sentinel.review_task`)
5. Nova researches using Tool Gateway (`web_search` , confirmed via `/tool/execute` , `/tool/list`)
6. Ghost stores validated knowledge (`core.ghost.ingest` , `core.ghost.search`)

7. EVA produces a final report (`mission.report`)

2. Runtime Stack

Layer	Handoff doc says (📄)	Actual code (✅)
Backend framework	FastAPI	⚠️ <code>stdlib http.server (ThreadingHTTPServer)</code> — deliberate, per <code>api.py</code> docstring: sandbox has no network access to install packages. Handler bodies are written to be portable to FastAPI later.
Database	PostgreSQL	? Unknown — <code>main()</code> takes a <code>db_path</code> argument (default <code>eva.db</code>), which is suggestive of SQLite but not confirmed without <code>core.py</code> / <code>memory/database.py</code> .
Memory/vector search	pgvector	? Unconfirmed. <code>ghost.search(query, mission_id)</code> exists but its retrieval method (keyword vs. vector) is unknown.
Agent framework	LangGraph / OpenAI Agents SDK	? No evidence yet of either in confirmed code. Agents (<code>Pixie</code> , <code>Nova</code> , <code>Sentinel</code> , <code>Ghost</code>) appear to be referenced as attributes/modules on <code>EVACore</code> , not via a third-party orchestration framework.
Frontend	React	⚠️ Actual deployed frontend is a single-file vanilla HTML/JS dashboard (no build step, no React) — confirmed from uploaded <code>index.html</code> .
Deployment	Docker Compose	? Unconfirmed — no Dockerfile or compose file has been provided yet.

Layer	Handoff doc says (📄)	Actual code (✅)
Entrypoint	—	✅ <code>python -m eva.api [port] [db_path] — default port 8000 , default db eva.db</code>

Guidance for Stage 2+: treat the "Handoff doc says" column as background intent only. Do not build against it where the "Actual code" column has a ✅ or ⚠️ entry — that reflects what's already running.

3. Repository Layout

📄 Proposed in handoff doc (not yet verified against a real file tree):

```
EVA/
  app/
    main.py
    core/ (eva.py, mission.py, planner.py)
    agents/ (pixie.py, nova.py, sentinel.py, ghost.py)
    memory/ (database.py, retrieval.py)
    security/ (permissions.py)
    tools/ (search.py, documents.py)
  tests/
  docs/
  docker-compose.yml
  README.md
```

✅ Confirmed to actually exist (from file status table + api.py imports):

```
eva/
  api.py           (confirmed, full content on file)
  core.py          <- imported as `from eva.core import
EVACore` — NOT YET SEEN
  models.py        <- imported as `from eva.models
import ...` — NOT YET SEEN
  tools/
    __init__.py    (exists per status table)
    base.py        (exists per status table)
    web_search.py  (exists — DuckDuckGo, no API key,
per status table)
```

```
gateway.py          (exists – "Sentinel-gated tool
execution", per status table)
```

? Open: whether `eva/agents/`, `eva/memory/`, `eva/security/` exist as separate modules, or whether `Pixie/Nova/Sentinel/Ghost` logic lives inside `core.py` and `tools/gateway.py`. `api.py` only ever calls `core.sentinel` and `core.ghost` – it never imports `Pixie` or `Nova` directly, which suggests those may be internal to `EVACore`, not top-level agent files. **This needs verification before Stage 2 scaffolds a directory tree.**

4. Agent Rules

📄 From handoff doc, not yet cross-checked against `core.py`:

Agent	Input	Output	Constraints
Pixie	Human request	Mission object	No system access
Nova	Approved research task	{Finding, Evidence, Confidence, Unknowns}	Must go through Tool Gateway (✅ confirmed pattern: <code>/tool/execute</code> routes through Sentinel-gated gateway)
Sentinel	Task	Authorization decision	Must approve tool use, external access, sensitive ops. Cannot modify EVA rules. ✅ Confirmed live: <code>core.sentinel.review_task(task, user_confirmed) → {decision, auth_level, reason}</code>
Ghost	Validated statement	Memory record	Stores {content, source, confidence, timestamp, classification}. ✅ Confirmed live: <code>core.ghost.ingest(mission_id, statement, source_id) → record + conflict list</code> ; <code>core.ghost.search(query, mission_id)</code>

5. Explicit Non-Goals (carried forward from handoff doc — still binding)

No sentence claims, no unrestricted autonomy, no self-modification, no consciousness layers, no large multi-agent swarms. These remain out of scope for the Research Node target.

6. Open Questions Blocking Full Confidence

1. What does `core.py` actually contain — does `EVACore.__init__` take `db_path`, and how are Pixie/Nova wired in?
2. What database engine backs `eva.db` — SQLite (suggested by filename) or something else?
3. What's inside `models.py` — full enum values for `AgentName`, `AuthLevel`, `TaskStatus`, and the `Mission / User / MemoryRecord` field definitions?
4. What does `tools/gateway.py` actually enforce — is Sentinel review mandatory for every tool call, or optional per `auth_level`?

Recommendation: Stage 2 (Database/Backend Agent) should not begin schema migration work until Q1–Q3 above are resolved, to avoid re-deriving a schema that conflicts with what `core.py` already implements.